SLIDESMANIA.COM

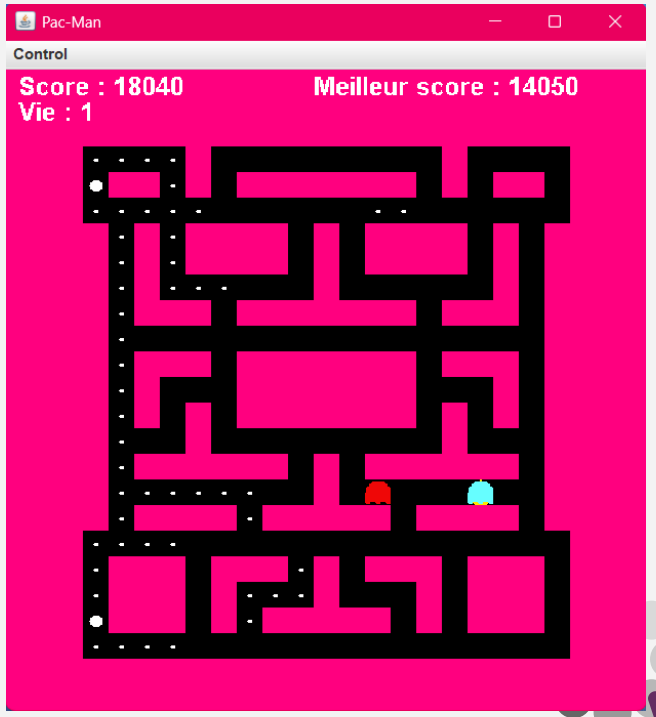
Artificially Intelligent People’s presentation

SLIDESMANIA.COM

Some statistics (over 25 tests)

	Average	Median	Minimum	Maximum
Score	4073	2700	270	18040
Time (ms)	32,31	29,10	16,55	56,35
Number of actions	825	464	30	3542

< 2000 pts	8
>= 2000 pts	17
Total	25



SLIDESMANIA.COM

findNextMove()

- Get the current state of the game,
- Analyze the environment,
- Run simulations to choose the best direction by assigning scores

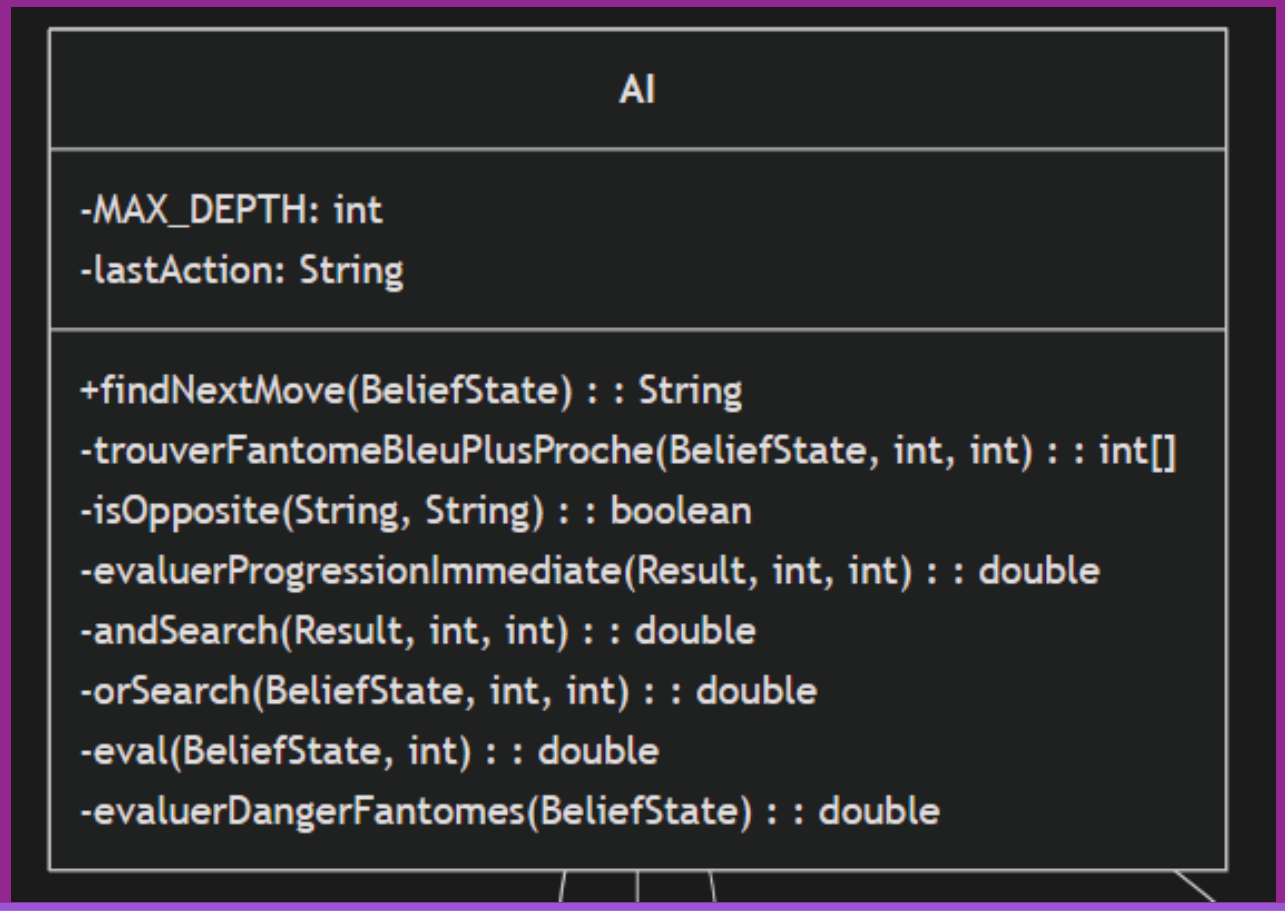
3 main phases:

1. Immediate Context analysis

2. AND-OR Search (depth 3)

3. Heuristic Adjustments

General Structure of the AI Class



SLIDESMANIA.COM

Phase 1: Context Analysis & Threat Detection

- Minimum distance to nearest gum
- Identify vulnerable ghost and nearest normal ghost
- Game state metrics :
  - Current gum count
  - Pac-Man position tracking

Ghost type differentiation :

- Scared ghosts are targets with bonus scoring
- Normal ghosts are threats requiring distance management

Special handling for ghosts at distance 1

SLIDESMANIA.COM

Phase 2: AND-OR search implementation

```
findNextMove()
     ↓
andSearch()   → Averages across
     ↓          possible belief
                states
orSearch()    → Selects best
     ↓          action branch
eval()        → Terminal state
                evaluation
                (depth 0 or game end)
```

Termination Conditions:

- Depth limit (=3) reached
- No gum remaining
- Pac-Man life ≤ 0

SLIDESMANIA.COM

OR SEARCH idea :

- Choice among multiple possible actions
- Returns best value
- Like Max in Minimax
  - PacMan is a rational player, so **he chooses an action that gives the best outcome**

AND-OR search

```java
private static double orSearch(BeliefState state, int depth, int gommesInitiales) {
    // conditions d'arret
    if (depth == 0 || state.getNbrOfGommes() == 0 || state.getLife() <= 0) {
        return eval(state, gommesInitiales);
    }

    Plans plans = state.extendsBeliefState();
    if (plans.size() == 0) return eval(state, gommesInitiales);

    double maxVal = Double.NEGATIVE_INFINITY;

    // recherche de la meilleure action
    for (int i = 0; i < plans.size(); i++) {
        Result res = plans.getResult(i);
        double val = andSearch(res, depth - 1, gommesInitiales);
        if (val > maxVal) {
            maxVal = val;
        }
    }

    return (maxVal == Double.NEGATIVE_INFINITY) ? -100000.0 : maxVal;
}
```

SLIDESMANIA.COM

AND-OR search

AND SEARCH idea :

- Average over all possible states
- Manages uncertainty
- Similar to expectation in expectiminimax
  - The game is unpredictable, when PacMan moves, sevaral things can happen
    - Since we don't know exactly what will happen, we take the average of all the possibilities.

```java
private static double andSearch(Result result, int depth, int gommesInitiales) {
    if (result.size() == 0) return Double.NEGATIVE_INFINITY;

    double totalScore = 0;
    int countValid = 0;

    for (BeliefState bs : result.getBeliefStates()) {
        double val = orSearch(bs, depth, gommesInitiales);

        // traitement different selon la valeur
        if (val > -50000) {
            totalScore += val;
            countValid++;
        } else {
            totalScore += val * 0.3;
            countValid++;
        }
    }

    return countValid > 0 ? totalScore / countValid : Double.NEGATIVE_INFINITY;
}
```

SLIDESMANIA.COM

Phase 2: AND-OR search implementation

- OR-SEARCH uses MAX because Pac-Man chooses the best possible action.
- AND-SEARCH uses AVERAGE because we must evaluate the expected results considering the game's uncertainty.

SLIDESMANIA.COM

Phase 3 :

1st heuristic : Gluttony

- +500 points for each gum eaten compared to the start of the turn. This makes him 'greedy'
- a bit of pressure: -100 points for each gum left on the map

Heuristic Adjustments & Decision Making

eval(…)

```
value = score
      + (gum_eaten * 500)
      - (remaining_gum * 100)
      - (gum_distance * weight)
      - ghost_danger
```

```
// bonus pour les gommes deja mangees
int gommesMangees = gommesInitiales - nbGommes;
score += gommesMangees * 500;

// penalite pour les gommes restantes
score -= nbGommes * 100;
```

SLIDESMANIA.COM

Phase 3 :

2nd heuristic : Dynamic Proximity

- penalize the distance from the nearest gum
- a dynamic weight :
  - 50 + (1500 / number of gums remaining)

Heuristic Adjustments & Decision Making

eval(…)

```
value = score
      + (gum_eaten * 500)
      - (remaining_gum * 100)
      - (gum_distance * weight)
      - ghost_danger
```

```
// calcul du poids de la distance
int poidsDistance = 50 + (1500 / (nbGommes + 1));

// penalite pour la distance a la gomme la plus proche
if (distGomme < Integer.MAX_VALUE) {
    score -= distGomme * poidsDistance;
}
```

SLIDESMANIA.COM

Phase 3 :

3rd heuristic : Security / Survival

- Bonus > fear ? Take some risk !
- Otherwise, escape

The evaluation corresponds to a weighted sum between the desire to eat, the urgency to finish the level and the fear of dying.

Heuristic Adjustments & Decision Making

eval(…)

```
value = score
      + (gum_eaten * 500)
      - (remaining_gum * 100)
      - (gum_distance * weight)
      - ghost_danger
```

```
// soustraction du danger des fantomes
score -= evaluerDangerFantomes(state);

return score;
```

SLIDESMANIA.COM

Pac-Man vs. the Ghosts

In findNextMove

Blue ghost proximity bonuses and...

- A penalty of -2000 will be applied if PacMan move away from the scared ghost.
- **Special case :** if a scared ghost is exactly at **distance 1** from PacMan (no matter where the other ghost is located), we add a **bonus of +3000** to force him to pursue him and eat him for the next action.

When dangerous ghosts are further away

Distance	Bonus
0	+8000
1	+5000
2	+3000

In evaluerDangerFantomes

Normal ghost danger penalties and...

- Special case if the 2 ghosts are at distance 1 from PacMan :
  - We reduce the danger (-200 instead of –1000) so that Pac-Man can escape and eat the gums.
    - Because he knows where the two ghosts are located, he can travel the map almost without fear.

Distance	Penalties
0	-100000
1	-1000
2	-300
3	-100
4	-10

Phase 3 : Heuristic Adjustments & Decision Making

SLIDESMANIA.COM

4th heuristic:
evaluerProgressionImmediate(Result result, int gommesInitiales, int distInitiale)

- **Gums eaten:** +4000 pts each
- **Distance improved:** +100 pts per square
- **Purpose:** Balance caution with action
- **Impact:** More aggressive, less hesitant

SLIDESMANIA.COM

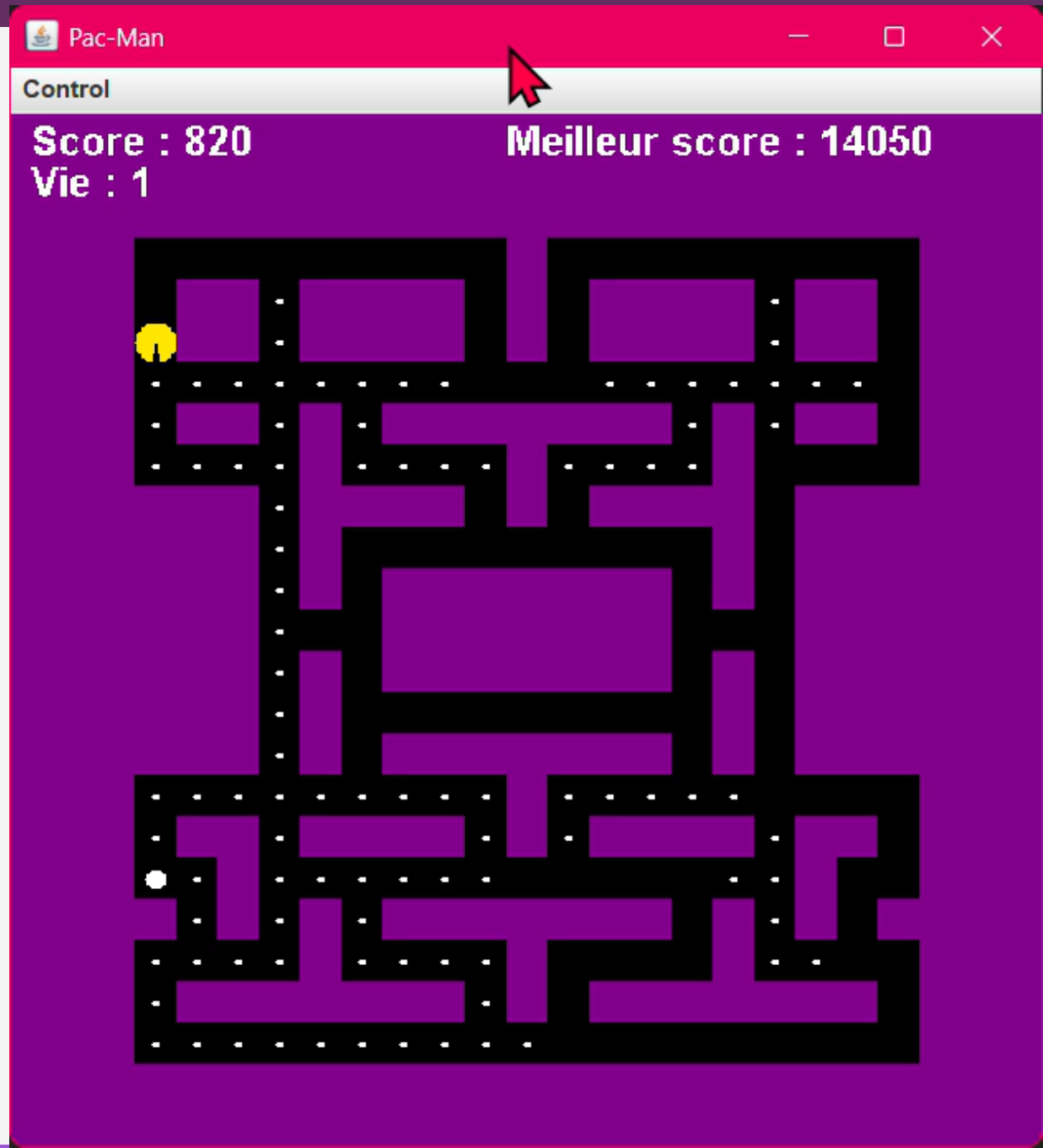
Issues we encountered / Areas for enhancement

SLIDESMANIA.COM

1) Find a good heuristic
2) Find the right balance between Pac-Man's fear andhunger
3) **The problem of Pac-Man refusing to eat an isolated gum when chased by two overlapping ghosts**
4) Solve the oscillation problem
5) Solve the cycle problem
6) Solve the Pac-Man immobility problem

SLIDESMANIA.COM

Video testing of our Pacman (fast forward)



SLIDESMANIA.COM

Solution “post-rendu” : no cycle and no oscillation

SLIDESMANIA.COM

Thank you for this project, it was fun and interesting !
Quite hard for us though…